DSEU
Crafting Excellence

Department of Computer Science Engineering

Course Syllabus and Credit Distribution 2024-25

Program : B.Tech in Computer Science Engineering

Department of Computer Science Engineering

Program: B.Tech in Computer Science Engineering

Credit Distribution Year 2024-25

Semester 3 NCrf Level : 5.0

S. No.	Course Type	Course Code	Course Title	Hours/ week	Total Credits
1.	DCC	CSE-4-301-T	Database Management Systems	3 (T)	3
2.	DCC	CSE-4-302-P	Database Management System Practice	2 (P)	1
3.	DCC	CSE-4-303-T	Algorithm Design and Analysis	3 (T)	3
4.	DCC	CSE-4-304-P	Algorithm Design and Analysis Practice	2 (P)	1
5.	DCC	CSE-4-305-T	Software Engineering	2 (T)	2
6.	AEC				2
7.	SEC				3
8.	MDC				1
9.	OEC & OTHERS				4
Total					**20**

Open/Other Elective Courses (OEC) for Semester III

S. No.	Course Type	Course Code	Course Title	Hours/ week	Total Credits
1.	OEC	CSE-4-306-T	Programming in Java	3 (T)	3
2.	OEC	CSE-4-307-P	Programming in Java Practice	2 (P)	1

Course Title: Database Management Systems			
Type of Course: DCC	**Level of Course: 5.0**	**Delivery Sub Type of the course: Theory**	
Course Code: CSE-4-301-T	**No. of credits: 3**	**T-P-S: 3-0-0**	**Learning hours: 45**

Prerequisite and Corequisite of Course: A basic understanding of computers, file systems, data types, data structures, and fundamental concepts of set theory is essential for effectively learning and applying the principles of Database Management Systems. These foundational topics support comprehension of data modeling, query formulation, and system architecture involved in DBMS.

Department: Computer Science Engineering

Course objectives

1. To understand the fundamental concepts and components of database systems, including data models, schema design, and the architecture of DBMS.
2. To explore various database design approaches and modeling techniques such as Entity-Relationship (ER) modeling and relational schema development.
3. To gain proficiency in essential DBMS functionalities, including data storage, indexing, normalization, transaction management, and recovery mechanisms.
4. To comprehend and implement concepts of database security, data integrity, access control, and concurrency control to ensure data reliability and protection.
5. To develop skills in writing and optimizing queries using Structured Query Language (SQL) for data retrieval, manipulation, and definition.
6. To acquire knowledge of advanced topics such as distributed databases, NoSQL databases, and database system scalability.
7. To apply normalization techniques to design efficient relational databases and reduce redundancy and inconsistency.
8. To implement hands-on mini-projects or lab-based exercises that simulate real-world database design and usage scenarios.
9. To understand the emerging trends in database technologies and their applications in domains such as big data analytics and cloud computing.

Course Content

Module /Unit	Topic	T	P	S
1	**Introduction to Databases and DBMS:** Basic concepts of Database, Database vs Files. DBMS Concept- Characteristics, applications, and challenges, Architecture. Codd's 12 rules. Data Independence, Roles and responsibilities of various database users, View of data, Data models - concept and types.	6		
2	**Entity Relationship Model:** ER Model Basic concepts, Design process and issues, E-R diagrams- Entities, attributes, relationships and relationship sets. Concepts of keys. Entity sets- Strong and weak, Cardinality, Mapping constraints, Extended E-R features - Generalization, Specialization, Aggregation, and Inheritance. ER diagram to tables Mapping	6		
3	**Relational Model and Relational Query Languages:** Introduction to the Relational Model, Structure of Relational Databases, Concepts such as Database Schema and Database Instance, Relations, Attributes, Tuples, Data Types and Domains, Keys including Primary, Candidate, Foreign, and Composite Keys, Constraints such as Domain, Key, Entity Integrity, and Referential Integrity, and Relationships including Unary, Binary, Ternary, with Participation and Cardinality. Relational Algebra: Fundamental Operators including Selection (σ), Projection (π), and Rename (ρ), Set Operations such as Union (∪), Intersection (∩), and Set Difference (−), Cartesian Product (×), Natural Join (⋈), Theta Join, and Equi Join, Division (÷) and its use cases, Extended Operators like Grouping, Aggregation, and Ungrouping, Query Construction involving syntax and semantics of expressions, and Relational Comparisons with Operator Precedence. Relational Calculus: Tuple Relational Calculus (TRC), Domain Relational Calculus (DRC), Syntax and Semantics of TRC and DRC Expressions, Examples of Queries in TRC and DRC, Comparison between Relational Algebra and Relational Calculus, and Computational Capabilities and Safety of Expressions..	9		
4	**Relational Database Design and Normalization:** Relational Database design Logical view of data, keys, integrity rules. Functional Dependency (FD) – definition, trivial and non-trivial FD, Inference Rules, Closure of FD set. Define Normalization, Type of Normalization – 1NF, 2NF, 3NF, BCNF, 4NF, 5NF	9		

5	**Structured Query Language (SQL)**: Basic concepts of SQL, Statements in SQL – Data Definition Language (DDL), Data Manipulation Language (DML), and Data Control Language (DCL), Structure creation and alteration, Defining constraints such as Primary Key, Foreign Key, Unique, Not Null, Check, and the use of the IN operator. Functions in SQL including Aggregate Functions, and Built-in Functions – Numeric, Date, and String Functions, Set Operations, Sub-queries, Use of various clauses such as GROUP BY, HAVING, and ORDER BY in SQL queries. Joins and its types – Inner Join, Outer Join, Left Join, Right Join, Full Outer Join, and Self Join, Nested Queries, Views and its types – Simple and Complex Views, and Application of Constraints within views.	9		
6	**Transaction Management, Indexing, and Emerging Trends:** Transaction Management concepts, States of Transaction, ACID properties, Serializability of transactions and concurrency control, Schedule, Deadlocks, Database Storage Structures and Indexing, Database Backup, Database Recovery, Database Administration and Security. Overview of NoSQL databases. Difference between SQL and NoSQL databases. Data Warehousing and OLAP. Big Data and Cloud Databases.	6		

Scheme of End Semester Examination: As per DSEU Regulation - 2(A), 2024 – End Term Examination: 70 Marks, Continuous Evaluation: 30 Marks, Total: 100 Marks.

Recommended Books and References:

1. Abraham Silberschatz, Henry F. Korth, S. Sudarshan, *Database System Concepts*, McGraw-Hill, 5th edition.
2. Raghurama Krishnan, Johannes Gehrke, *Database Management Systems*, TATA McGraw-Hill, 3rd edition.
3. Elmasri, Ramez, and Navathe, Shamkant B., *Fundamentals of Database Systems*, Pearson Education, latest edition.
4. C. J. Date, A. Kannan, S. Swami Nadhan, *An Introduction to Database Systems*, Pearson, 8th edition.

Learning Outcomes:

1. Students will be able to design efficient and normalized databases, create ER diagrams, and translate them into relational schemas.
2. Students will be able to apply normalization techniques to remove duplicates and improve database design.
3. Students will be able to apply DBMS concepts to real-world database management and design.
4. Students will be able to execute SQL queries for effective manipulation and management of databases.

Hyperlinks of suggested e-Resources:

1. Coursera - Introduction to Databases: https://www.coursera.org/learn/introduction-to-databases
2. NPTEL - Database Management System: https://onlinecourses.nptel.ac.in/noc22_cs91/preview
3. Coursera - Introduction to Relational Databases (RDBMS): https://www.coursera.org/learn/introduction-to-relational-databases

Pedagogical approach:

- Classroom teaching using traditional board or audio-visual medium as and when necessary.
- Flip classroom methodology to encourage active learning and student participation.
- Quizzes and presentations conducted as per the teacher's discretion, based on the topic and student engagement.

Additional information (if any)

Course Title: Database Management System Practice			
Type of Course: DCC	**Level of Course: 5.0**	**Delivery Sub Type of the course:**	**Practical**
Course code: CSE-4-302-P	**No. of credits: 1**	**T-P-S: 0-2-0**	**Learning hours: 30**

Prerequisite and Corequisite of Course: Students should have a basic understanding of computer skills, must be concurrently enrolled in or have completed the DBMS theory course, and should be familiar with the Linux operating system and command-line interface.

Department: BTech in Computer Science Engineering

Course objectives

1. To enable students to analyze real-world scenarios and design corresponding Entity-Relationship (ER) diagrams.
2. To develop the ability to convert ER diagrams into normalized relational schemas.
3. To provide hands-on experience in creating, querying, updating, and managing databases using Structured Query Language (SQL).
4. To familiarize students with key constraints, joins, nested queries, views, and transaction control in SQL.
5. To introduce students to the concepts of NoSQL databases and provide practical exposure to MongoDB.
6. To develop skills in using database management tools and interfaces in Linux-based environments.
7. To enhance students' understanding of database design through lab exercises, mini-projects, and assignments.

Course content

Program No.	Query Statement	T	P	S
1	1) To install and configure MongoDB and/or MySQL and/or PostgreSQL on Linux or Windows operating systems. 2) To create user roles such as Admin, Data Entry Operator, and Data Analyst in a DBMS and assign appropriate privileges such as SELECT, INSERT, UPDATE, and DELETE to each role.		2	
2	3) To design ER diagrams for systems like University Management/Library Management / Hospital Management / Banking System using tools such as draw.io, MySQL Workbench, MS Word, or LibreOffice Draw. a) Draw the ER diagram using generalization and specialization, and define the relationships. b) Map ER diagrams into relational database tables.		4	
3	4) To decompose the tables created in Program No.2 into 1NF tables by ensuring that each column contains only atomic (single) values and there are no duplicate rows. 5) To decompose the tables in 1NF created as above into 2NF tables to eliminate partial dependencies. 6) To decompose the tables in 2NF created as above into 3NF tables by ensuring to remove transitive dependencies. 7) To decompose the tables in 3NF created as above into BCNF tables to ensure all determinants are candidate keys.		6	
4	8) To create tables using SQL and apply constraints such as PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, and CHECK. 9) Write a SQL statement for implementing ALTER, UPDATE and DELETE operations. 10) Write the queries to implement the different types of joins: inner join, outer join, left outer join, right outer join, full outer join. 11) Write the SQL statements using the built in functions (max, min, avg, count etc.), SET functions and string functions. 12) Write the queries to implement the concept of Integrity constraints. 13) Write the queries to create, update, delete views and triggers.		10	
5	14) To implement a basic banking transaction system demonstrating ACID properties (Atomicity, Consistency, Isolation, Durability). 15) To manage transactions using COMMIT, ROLLBACK, and SAVEPOINT commands.		8	

	16) To design and document a database backup strategy for a large transactional system. 17) To implement a simple user authentication system using database tables and queries.			

Scheme of End Semester Examination: As per DSEU Regulation - 2(A), 2024 – End Term Examination: 70 Marks, Continuous Evaluation: 30 Marks, Total: 100 Marks.

Recommended Books and References:

1) Database Management Systems, Raghurama Krishnan, Johannes Gehrke, TATA McGraw Hill, 3rd Edition
2) Database System Concepts, Abraham Silberschatz, Henry F. Korth, S. Sudarshan, McGraw-Hill, 5th Edition (2005)
3) Fundamentals of Database Systems, Elmasri Navathe, Pearson Education
4) An Introduction to Database Systems, C.J. Date, A. Kannan, S. Swami Nadhan, Pearson, 8th Edition

Learning Outcomes:

Upon completion of the course, students will be able:

1) Design robust database schemas by analyzing real-world requirements and applying techniques like Entity-Relationship (ER) modeling, normalization, and schema design principles.
2) Write efficient and optimized SQL queries to manipulate, retrieve, and manage data, ensuring effective use of relational database management systems (RDBMS).
3) Implement transactions in SQL that fully satisfy the ACID properties (Atomicity, Consistency, Isolation, Durability), ensuring reliable and consistent database operations.
4) Enforce data security measures by managing user access, implementing authentication and authorization protocols, and ensuring the protection of sensitive data within the database.
5) Utilize advanced SQL features such as joins, subqueries, views, and stored procedures to solve complex data manipulation problems.
6) Apply database normalization techniques to reduce redundancy and improve data integrity, including converting tables into First, Second, Third, and Boyce-Codd Normal Forms (1NF, 2NF, 3NF, BCNF).
7) Understand and apply database indexing to optimize query performance and improve data retrieval times in large datasets.
8) Design and implement a backup strategy for databases, ensuring data availability and disaster recovery in production environments.
9) Work with database management tools such as MySQL Workbench, MongoDB Compass, or others, to model, manage, and visualize database structures.

Hyperlinks of suggested e-Resources:

1) Coursera - Introduction to Databases https://www.coursera.org/learn/introduction-to-databases
2) NPTEL - Data Base Management System https://onlinecourses.nptel.ac.in/noc22_cs91/preview
3) Coursera - Introduction to Relational Databases (RDBMS) https://www.coursera.org/learn/introduction-to-relational-databases

Pedagogical approach:

- Clarify the Objective: Before starting, the teacher should explain the exact interpretation of the task in the syllabus. This would involve a clear understanding of the problem, which helps students grasp the purpose behind the commands and queries they will be working with.
- Theoretical Knowledge: The teacher must explain all the theoretical concepts, including database design, ER diagrams, normalization, and any commands relevant to the query, such as SELECT, INSERT, JOIN, etc. Additionally, variations of these commands in different databases (like MySQL, MongoDB, PostgreSQL) should be taught, emphasizing the syntax differences.
- Command Writing and Verification: After providing the theoretical background, students should attempt writing the commands or queries on their own. The teacher should ensure they understand the logic behind the command and provide feedback on the syntax or logical errors before allowing them to execute it on the system.
- Execution and Testing: Once students are confident in writing the commands, they can execute them on the system. There should be enough data in the database (multiple tables with enough rows) so that various tests and queries can be performed. This ensures that the student understands the scope of the query and how to validate results.
- Feedback and Correction: After execution, the teacher should guide students to compare results with the expected output, identify any mistakes, and correct them both in their notebooks and on the system. This process reinforces understanding and helps students become more confident in their abilities.
- Hands-On Experience: Finally, the teacher should ensure that every student practices commands several times across different types of queries, which helps in building proficiency in real-world database management tasks.

Additional information (if any)

Course Title: Algorithm Design and Analysis			
Type of Course: DCC	**Level of Course: 5.0**	**Delivery Sub Type of the course:**	**Theory**
Course code: CSE-4-303-T	**No. of credits: 3**	**T-P-S: 3-0-0**	**Learning hours: 45**

Pre-requisite and Co-requisite of Course:

Data Structures and Algorithms (DSA): Understanding of fundamental data structures (arrays, linked lists, stacks, queues, trees, graphs) and algorithmic techniques (sorting, searching, recursion, etc.) is essential for efficient database design and query optimization.

Programming Language Proficiency (C/C++): Ability to write basic programs, understand control structures, and handle input/output operations is necessary for implementing procedural logic in triggers, stored procedures, and interfacing with databases.

Department: Computer Science Engineering

Course objectives

1. To Understand algorithm design principles and efficiency analysis.
2. To Analyze time and space complexities using asymptotic notations.
3. To Learn key paradigms: divide and conquer, greedy, and dynamic programming.
4. To Explore graph algorithms and their real-world applications.
5. To Study NP-completeness and approximation techniques.
6. To Compare and evaluate algorithm performance across problem domains.
7. To Enhance problem-solving skills with efficient algorithmic solutions.

Course Content

Module/ Unit	Topic	T	P	S
1	**Introduction to Algorithms and Complexity:** Algorithms: Definition, Characteristics, Importance, Algorithm vs. Program, Time Complexity and Space Complexity, Asymptotic Notations: Big-O, Big-Ω, Big-Θ, o-notation, ω-notation, Examples of Complexity Analysis, Recursion and Recurrence: Recursion Basics, Recursive Algorithms, Master Theorem, Substitution Method, Recursion Trees, Solving Recurrence Relations	7		
2	**Divide and Conquer Strategy** : Overview of Divide and Conquer Strategy, Revisiting Binary Search, Heap Sort, Quick Sort, Merge Sort, Radix Sort, Time and Space Complexity Analysis, Strassen's Matrix Chain Multiplication, Medians and Order Statistics: Minima, Maxima	7		
3	**Dynamic Programming:** Difference Between Divide and Conquer and Dynamic Programming, Matrix Chain Multiplication, Elements of Dynamic Programming: Optimal Substructure, Overlapping Subproblems, Memorization, Longest Common Subsequence (LCS), 0/1 Knapsack Problem, Optimal BST	7		
4	**Greedy Algorithms:** Characteristics of Greedy Approach, Activity Selection Problem, Huffman Coding, Fractional Knapsack Problem, Revisiting Minimum Spanning Tree: Prim's Algorithm, Kruskal's Algorithm	6		
5	**Graph Algorithms:** Data Structures for Disjoint Sets: Disjoint Set Operations, Connected Components, Linked Lists for Disjoint Sets, Revisiting BFS and DFS, Topological Sorting, Strongly Connected Components, Single-Source Shortest Paths: Dijkstra's Algorithm, Bellman-Ford Algorithm, All-Pair Shortest Paths: Floyd-Warshall's Algorithm	8		
6	**String Matching Algorithms**: Naïve String Matching, Rabin-Karp Algorithm, String Matching with Finite Automata, Knuth-Morris-Pratt Algorithm (KMP)	4		
7	**Computational Complexity**: Complexity Classes: P, NP, NP-Hard, NP-Complete, Polynomial Time Verification, Reducibility, Vertex Cover Problem, Travelling Salesman Problem	6		

Scheme of End Semester Examination: As per DSEU Regulation - 2(A), 2024 – End Term Examination: 70 Marks, Continuous Evaluation: 30 Marks, Total: 100 Marks.

Recommended Books and References:

1) Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, MIT Press, 4th Edition
2) Algorithms, Robert Sedgewick, Kevin Wayne, Addison-Wesley, 4th Edition
3) Algorithm Design, Jon Kleinberg, Éva Tardos, Pearson Education, 1st Edition
4) The Design and Analysis of Algorithms, Anany Levitin, Pearson Education, 3rd Edition
5) Algorithms Unlocked, Thomas H. Cormen, MIT Press, 1st Edition
6) Algorithms in a Nutshell: A Practical Guide, George T. Heineman, Gary Pollice, Stanley Selkow, O'Reilly Media, 2nd Edition
7) Data Structures and Algorithm Analysis in C++, Mark Allen Weiss, Pearson Education, 4th Edition

Learning outcomes

Upon successful completion of this course, students will be able to:

1. Analyze the efficiency of algorithms by evaluating their time and space complexity using Big-O, Big-Ω, and Big-Θ notations.
2. Apply various algorithmic paradigms including brute force, divide and conquer, dynamic programming, greedy algorithms, and backtracking to design optimal solutions for computational problems.
3. Design and implement efficient algorithms for common problems in sorting, searching, graph theory, and string manipulation, ensuring they meet both correctness and efficiency criteria.
4. Optimize algorithms by applying advanced techniques such as memorization, dynamic programming, and parallel processing to improve performance.
5. Solve complex real-world problems using dynamic programming, greedy strategies, and divide and conquer approaches, making informed choices based on the problem type and constraints.
6. Identify and classify NP-complete problems and explore approximation algorithms, heuristics, and problem-specific strategies to handle intractable problems.
7. Evaluate and compare the performance of different algorithms in real-world scenarios through both theoretical analysis and empirical testing, including analyzing trade-offs between time complexity and space complexity.
8. Demonstrate strong algorithmic thinking and problem-solving skills, approaching problems with structured methodologies and applying appropriate algorithmic solutions to new and unfamiliar challenges.
9. Develop and implement algorithms in real-world programming environments, ensuring correctness, efficiency, and scalability for large datasets and complex problems.
10. Communicate algorithmic solutions effectively, providing clear explanations of algorithms' steps, advantages, and trade-offs, and using appropriate notation and terminology.

Hyperlinks of suggested e-Resources:

1. MIT OpenCourseWare - Introduction to Algorithms: https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-006-introduction-to-algorithms-fall-2011/
2. Coursera - Algorithms Specialization: https://www.coursera.org/specializations/algorithms
3. GeeksforGeeks - Fundamentals of Algorithms: https://www.geeksforgeeks.org/fundamentals-of-algorithms/
4. Visualgo - Algorithm Visualization: https://visualgo.net/en
5. NPTEL - Analysis and Design of Algorithms: https://onlinecourses.nptel.ac.in/noc22_cs91/preview

Pedagogical approach

- Lecture-based teaching for theoretical concepts.
- Problem-solving sessions to apply algorithms.
- Use of visualization tools like VisuAlgo.
- Hands-on coding assignments.
- Flipped classroom for discussions and exercises.
- Group activities for collaborative learning.
- Gamification through competitive coding challenges.
- Case studies on real-world algorithm applications.

Additional information (if any)

Course Title: Algorithm Design and Analysis Practice			
Type of Course: DCC	**Level of Course: 5.0**	**Delivery Sub Type of the course:**	**Practical**
Course code: CSE-4-304-P	**No. of credits: 1**	**T-P-S: 0-0-2**	**Learning hours: 30**

Pre-requisite: Data Structures and Algorithms (DSA): Knowledge of fundamental data structures such as arrays, linked lists, stacks, queues, trees, and graphs, along with basic algorithms for searching and sorting.
Co-requisite: Programming Language Knowledge (C/C++): Proficiency in a programming language, particularly C or C++, as the implementation and analysis of algorithms will require coding skills.

Department: Computer Science Engineering

Course objectives

1. To implement and analyze fundamental algorithms, including those for searching, sorting, and graph traversal.
2. To study and understand time and space complexities of algorithms using Big-O notation and related complexity classes.
3. To apply key algorithm design paradigms such as divide and conquer, greedy algorithms, dynamic programming, and backtracking.
4. To solve real-world problems by selecting and applying the most appropriate and efficient algorithmic approaches.
5. To work on hands-on coding assignments to solidify understanding and enhance problem-solving capabilities.
6. To utilize algorithm visualization tools to observe and analyze the behavior and efficiency of algorithms.
7. To develop skills in comparing the performance of algorithms, optimizing solutions, and justifying choices based on problem requirements.
8. To understand the theoretical foundations behind algorithm design and its real-world applications.

Course Content

Program No.	Program Statement	T	P	S
1	To implement the Binary Search algorithm and measure its execution time for different input sizes, then compare the results with theoretical time complexity using Big-O notation.		**2**	
2	To implement the Bubble Sort and Insertion Sort algorithms, measure their execution time for varying input sizes, analyze their time complexities using Big-O notation, and compare the experimental results with the theoretical time complexities.		2	
3	To implement the Merge Sort and Quick Sort algorithms, measure their execution time for varying input sizes, analyze their time complexities using Big-O notation, and compare the experimental results with the theoretical time complexities.		2	
4	To implement Strassen's Matrix Chain Multiplication (MCM) algorithm and analyze its time complexity.		2	
5	To implement the Longest Common Subsequence (LCS) problem and analyze its time complexity.		2	
6	To implement the Knapsack problem algorithms: a) 0/1 Knapsack Problem (where each item can either be included or excluded from the knapsack) and analyze its time complexity. b) Fractional Knapsack Problem (where items can be divided and fractions of them can be included in the knapsack) and analyze its time complexity.		2	
7	To implement the Huffman Coding algorithm for data compression and analyze its time complexity.		2	
8	To implement the Activity Selection Problem and analyze its time complexity.		2	
9	To implement Dijkstra's Algorithm for finding the shortest path in a graph and analyze its time complexity.		2	
10	To implement the Bellman-Ford Algorithm for finding shortest paths and handling graphs with negative weights and analyze its time complexity.		2	
11	To implement the Floyd-Warshall Algorithm for finding all-pairs shortest paths and analyze its time complexity.		2	
12	To implement the Topological Sort algorithm for directed acyclic graphs (DAGs) and analyze its time complexity.		2	
13	To implement the Naïve String Matching algorithm and analyze its time complexity.		2	
14	To implement the Rabin-Karp Algorithm for string searching and analyze its time complexity.		2	
15	To implement the Knuth-Morris-Pratt (KMP) Algorithm for efficient string matching and analyze its time complexity.		2	

Scheme of End Semester Examination: As per DSEU Regulation - 2(A), 2024 – End Term Examination: 70 Marks, Continuous Evaluation: 30 Marks, Total: 100 Marks.

Recommended Books:

1. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, MIT Press, 4th Edition, 2022
2. Algorithms, Robert Sedgewick, Kevin Wayne, Addison-Wesley, 4th Edition, 2014.
3. The Design and Analysis of Algorithms, Anany Levitin, Pearson Education, 3rd Edition, 2011.
4. Algorithms Unlocked, Thomas H. Cormen, MIT Press, 1st Edition, 2013.
5. Algorithms in a Nutshell: A Practical Guide, George T. Heineman, Gary Pollice, Stanley Selkow, O'Reilly Media, 2nd Edition, 2016.
6. Data Structures and Algorithm Analysis in C++, Mark Allen Weiss, Pearson Education, 4th Edition, 2014.
7. The C Programming Language, Brian W. Kernighan, Dennis M. Ritchie, Prentice Hall, 2nd Edition, 1988.
8. C++ Programming: From Problem Analysis to Program Design, D.S. Malik, Cengage Learning, 6th Edition, 2017.

Learning outcomes

By the end of the course, students will be able to:

1. Implement and optimize fundamental algorithms for a variety of problems, including sorting, searching, graph algorithms, and dynamic programming techniques.
2. Analyze and evaluate the efficiency of algorithms, considering time and space complexity, and apply Big-O notation to different algorithmic scenarios.
3. Apply a range of algorithmic paradigms such as divide and conquer, greedy algorithms, dynamic programming, and backtracking to solve computational problems efficiently.
4. Design and implement algorithms for real-world problems, such as the knapsack problem, shortest path algorithms, and sorting, using appropriate algorithmic strategies.

Hyperlinks of suggested e-Resources:

1. MIT OpenCourseWare - Introduction to Algorithms: https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-006-introduction-to-algorithms-fall-2011/
2. Coursera - Algorithms Specialization: https://www.coursera.org/specializations/algorithms
3. GeeksforGeeks - Fundamentals of Algorithms: https://www.geeksforgeeks.org/fundamentals-of-algorithms/
4. Visualgo - Algorithm Visualization: https://visualgo.net/en
5. NPTEL - Analysis and Design of Algorithms: https://onlinecourses.nptel.ac.in/noc22_cs91/preview

Pedagogical approach

- Hands-on coding and implementation of algorithms.
- Use of algorithm visualization tools to enhance understanding.
- Use of competitive coding platforms for practice and improvement.
- Emphasis on algorithmic optimization and performance analysis.
- Encouraging self-learning through exploration of advanced topics.

Additional information (if any)

Course Title: Software Engineering			
Type of Course: DCC	**Level of Course: 5.0**	**Delivery Sub Type of the course:**	**Theory**
Course code: CSE-4-305-T	**No. of credits: 2**	**T-P-S: 2-0-0**	**Learning hours: 30**

Pre-requisite and Co-requisite of Course:

1. Programming Language Knowledge (C/C++/Java/Python) – Students should be comfortable writing and understanding code.
2. Data Structures and Algorithms (DSA) – Essential for understanding efficient problem-solving and software performance.
3. Object-Oriented Programming (OOP) – Core to understanding modern software design principles and modular development.

Department: Computer Science Engineering

Course objectives:

1. To understand the fundamental concepts, principles, and practices of software engineering.
2. To introduce a systematic, structured, and disciplined approach to software development.
3. To familiarize students with various software development life cycle (SDLC) models and process frameworks.
4. To explore software requirement analysis, design methodologies, coding practices, testing strategies, and maintenance.
5. To understand and apply software metrics and quality assurance techniques for effective project management.
6. To equip students with tools and techniques for planning, estimating, and controlling software projects.

Course Content

Module/ Unit	Topic	T	P	S
1	Introduction to Software Engineering, Software Components, Software Characteristics, Software Crisis, Software Engineering Processes, II Similarity and Differences from Conventional Engineering Processes, Software Quality Attributes. Software Development Life Cycle (SDLC) Models: Water-Fall Model, Prototype Model, Spiral Model, Evolutionary Development Models, Iterative Enhancement Models.	6		
2	Software Requirement Specification: Requirements Elicitation Techniques, Requirements analysis, Models for Requirements analysis, requirements specification, requirements validation.	6		
3	Software Design: Basic Concept of Software Design, Architectural Design, Low Level Design: Modularization, Design Structure Charts, Pseudo Codes, Flow Charts, Coupling and Cohesion Measures, Design Strategies: Function Oriented Design, Object Oriented Design, Top-Down and Bottom-Up Design. Software Measurement and Metrics: Size Measures: Function Point (FP), Estimation of Various Parameters such as Cost, Efforts, Schedule/Duration, Constructive Cost Models (COCOMO).	8		
4	Software Testing: Testing Objectives, Unit Testing, Integration Testing, Acceptance Testing, Regression Testing, Testing for Functionality and Testing for Performance, Top-Down and Bottom-Up Testing Strategies: Test Drivers and Test Stubs, Functional Testing (Black Box Testing), Structural Testing (White Box Testing): Cyclomatic Complexity Measures, Control Flow Graphs, Test Data Suit Preparation, Alpha and Beta Testing of Products. Static Testing Strategies: Formal Technical Reviews (Peer Reviews), Walk Through, Code Inspection, Compliance with Design and Coding Standards.	6		
5	Software Maintenance and Software Project Management Software as an Evolutionary Entity, Need for Maintenance, Categories of Maintenance: Preventive, Corrective and Perfective Maintenance, Cost of Maintenance, Software Re-Engineering, Reverse Engineering. Software Configuration Management Activities, Change Control Process, Software Version Control, An Overview of CASE Tools Software Risk Analysis and Management.	4		

Scheme of End Semester Examination: As per DSEU Regulation - 2(A), 2024 – End Term Examination: 70 Marks, Continuous Evaluation: 30 Marks, Total: 100 Marks.

Recommended Books and References:

1. Software Engineering – A Practitioner's Approach, R. S. Pressman, McGraw Hill International Edition, Latest edition
2. Software Engineering – by K.K.Aggarwal and Yogesh Singh, New Age Publisher, Revised 2nd Edition.
3. Fundamentals of Software Engineering, Rajib Mall, PHI Publication.

4. Software Engineering: A Precise Approach, Pankaj Jalote, Wiley India, 2010
5. Software Engineering: With Open Source and GenAI, Pankaj Jalote, Wiley India, 2025 (2nd Edition)

Learning outcomes

By the end of this course, students will be able to:

1. Apply software engineering principles and practices to design, develop, test, and implement reliable software systems.
2. Use appropriate software development models and methodologies (e.g., Waterfall, Agile) for different project scenarios.
3. Analyze user requirements and translate them into formal specifications.
4. Employ testing techniques to ensure software correctness and quality
5. Manage software projects effectively to ensure timely delivery, quality assurance, and risk mitigation.
6. Work collaboratively in teams using version control and software management tools.

Hyperlinks of suggested e-Resources:

1. Coursera – Software Engineering Specialization (University of Alberta) https://www.coursera.org/specializations/software-design-architecture
2. MIT OpenCourseWare – Software Engineering for Internet Applications – https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-171-software-engineering-for-internet-applications-spring-2006/
3. NPTEL – Software Engineering by Prof. Rajib Mall (IIT Kharagpur) – https://nptel.ac.in/courses/106105087

Pedagogical approach :

- Lecture-based teaching for theoretical concepts.
- Problem-solving sessions to apply algorithms.
- Use of visualization tools like VisuAlgo.
- Hands-on coding assignments.
- Flipped classroom for discussions and exercises.
- Group activities for collaborative learning.
- Gamification through competitive coding challenges.
- Case studies on real-world algorithm applications.

Additional information (if any)

Department of Computer Science Engineering

Program: B.Tech in Computer Science Engineering

Credit Distribution Year 2024-25

Semester 4 NCrF Level : 5.0

S. No.	Course Type	Course Code	Course Title	Hours/ week	Total Credits
1.	DCC	CSE-4-401-T	Operating Systems	3	3
2.	DCC	CSE-4-402-P	Operating Systems Practice using Unix/Linux	2	1
3.	DCC	CSE-4-403-T	Web Development	2	2
4.	DCC	CSE-4-404-P	Web Development Practice	2	1
5.	DCC	CSE-4-405-P	Python Programming Practice	2	1
6.	DSE	ELECTIVES-1			4
7.	AEC				2
8.	SEC				2
9.	OEC & OTHERS				4
Total					**20**

Department Specific Electives (DSE) (ELECTIVES-I)

S. No.	Course Type	Course Code	Course Title	Hours/week	Total Credits
1.	DSE	CSE-4-406-T	Computational Mathematics	4 (T)	4
2.	DSE	CSE-4-407-T	Discrete Mathematical Structures	4 (T)	4

Course Title: Operating Systems				
Type of Course: DCC	**Level of Course: 5.0**	**Delivery Sub Type of the course:**		**Theory**
Course code: CSE-4-401-T	**No. of credits: 3**	**T-P-S: 3-0-0**		**Learning hours: 45**
Prerequisite and Corequisite of Course: Basic Understanding of Computers, Number Systems, Basic Mathematics				
Department: Computer Science Engineering				

Course objectives

1. To develop an in-depth understanding of the fundamental concepts, structure, and functionalities of modern operating systems.
2. To understand how operating systems manage hardware resources such as CPU, memory, storage, and I/O devices
3. To learn about process management, scheduling, synchronization, and inter-process communication.
4. To study memory management techniques including paging, segmentation, and virtual memory.
5. To explore file systems, storage management, and security features in operating systems.
6. To understand the design and implementation of operating system components.

Course Content

Module/ Unit	Topic	T	P	S
1	**Introduction**: Concept of Operating Operating systems, Generations of Operating Systems, Types of Operating Systems, OS Services, System Calls, Structure of an OS – Layered, Monolithic, Microkernel Operating Systems, Concept of Virtual Machine. Case study on Linux/UNIX and Windows Operating Systems.	4		
2	**Processes**: Definition, Process Relationship, Different states of a Process, Process State transitions, Process Control Block (PCB), Context switching. **Thread**: Definition, Various states, Benefits of threads, Types of threads, Concept of multithreading. **Process Scheduling**: Foundation and Scheduling objectives, Types of Schedulers, Scheduling criteria: CPU utilization, Throughput, Turnaround Time, Waiting Time, Response Time; Scheduling algorithms: Pre-emptive and Non pre-emptive, FCFS, SJF, RR; Multiprocessor scheduling: Real Time scheduling: RM and EDF	9		
3	**Inter-process Communication:** Critical Section, Race Conditions, Mutual Exclusion, Hardware Solution, Strict Alternation, Peterson's Solution, The Producer Consumer Problem, Semaphores, Event Counters, Monitors, Message Passing, Classical IPC Problems: Reader's & Writer Problem, Dining Philosopher Problem etc.	9		
4	**Deadlocks:** Definition, Necessary and sufficient conditions for Deadlock, Deadlock Prevention, Deadlock Avoidance: Banker's algorithm, Deadlock detection and Recovery.	6		
5	**Memory Management:** Basic concept, Logical and Physical address map, Memory allocation: Contiguous Memory allocation– Fixed and variable partition– Internal and External fragmentation and Compaction; Paging: Principle of operation Page allocation Hardware support for paging, Protection and sharing, Disadvantages of paging. **Virtual Memory:** Basics of Virtual Memory – Hardware and control structures – Locality of reference, Page fault , Working Set , Dirty page/Dirty bit – Demand paging, Page Replacement algorithms: Optimal, First in First Out (FIFO), Second Chance (SC), Not recently used (NRU) and Least Recently used (LRU)	9		
6	**I/O Hardware:** I/O devices, Device controllers, Direct memory access Principles of I/O Software: Goals of Interrupt handlers, Device drivers, Device independent I/O software, Secondary-Storage Structure: Disk structure, Disk scheduling algorithms **File Management:** Concept of File, Access methods, File types, File operation, Directory structure, File System structure, Allocation methods (contiguous, linked, indexed), Free-space management (bit vector, linked list, grouping), directory implementation (linear list, hash table), efficiency and performance. File systems: FAT, FAT32, Ext3/4, ZFS and BtrFS. **Disk Management:** Disk structure, Disk scheduling – FCFS, SSTF, SCAN, C-SCAN, Disk reliability, Disk formatting, Boot-block, Bad blocks	8		

Scheme of End Semester Examination: As per DSEU Regulation - 2(A), 2024 – End Term Examination: 70 Marks, Continuous Evaluation: 30 Marks, Total: 100 Marks.

Recommended Books and References:

1. Operating System Concepts Essentials, Avi Silberschatz, Peter Galvin, Greg Gagne, Wiley Asia Student Edition, 9th Edition
2. Operating Systems: Internals and Design Principles, William Stallings, Prentice Hall of India, 5th Edition
3. Operating Systems: A Modern Perspective, Gary J. Nutt, Addison-Wesley, 2nd Edition
4. Design of the Unix Operating Systems, Maurice Bach, Prentice-Hall of India, 8th Edition

Learning outcomes:

Upon successful completion of this course, students will be able to:

1. Demonstrate an in-depth understanding of core operating system concepts including architecture, functions, and services.
2. Analyze and explain the concepts of processes and threads, including interprocess communication mechanisms and synchronization.
3. Evaluate deadlock conditions and apply prevention, avoidance, and recovery techniques.
4. Understand and apply memory management techniques such as paging, segmentation, and virtual memory.
5. Comprehend the structure and management of I/O hardware, disk scheduling, and file systems.
6. Design and implement multithreaded applications using appropriate synchronization constructs.
7. Analyze and optimize system performance metrics such as CPU utilization, throughput, turnaround time, waiting time, response time, and identify system bottlenecks.

Hyperlinks of suggested e-Resources:

1. Operating System Engineering – Massachusetts Institute of Technology (MIT OpenCourseWare): https://ocw.mit.edu/courses/6-828-operating-system-engineering-fall-2012/
2. Operating System Fundamentals – NPTEL (Instructor: Prof. Niket Agrawal, IIT Roorkee): https://onlinecourses.nptel.ac.in/noc24_cs108/preview
3. Operating Systems – NPTEL (Instructor: Prof. Sorav Bansal, IIT Delhi): https://onlinecourses.nptel.ac.in/noc24_cs80/preview
4. Introduction to Operating Systems – Coursera (Offered by Akamai Technologies): https://www.coursera.org/learn/akamai-operating-systems

Pedagogical approach:

- **Classroom Instruction**: Conceptual discussions and explanations will be delivered using traditional board teaching or audiovisual media, depending on the nature of the topic and the need for visual aids.
- **Flipped Classroom Approach**: Students will engage with course material through pre-recorded lectures, readings, and resources, and use classroom time for active learning activities, discussions, and problem-solving exercises.
- **Quizzes and Presentations**: Regular quizzes will be conducted to assess student understanding, while presentations will encourage students to demonstrate their learning and critical thinking abilities on assigned topics.
- **Adaptive Learning Strategies**: The teaching methods will be adjusted based on the needs of the students, incorporating additional strategies such as group discussions, case studies, or practical sessions as deemed necessary by the instructor.

Additional information (if any)

Course Title: Operating System Practice Using Unix/Linux			
Type of Course: DCC	**Level of Course: 5.0**	**Delivery Sub Type of the course: Practical**	
Course Code: CSE-4-402-P	**No. of credits: 1**	**T-P-S: 0-2-0**	**Learning hours: 30**

Prerequisite and Corequisite of Course: Basic understanding of computers, Operating systems and command line terminal

Department: Computer Science Engineering

Course objectives

1. To install and configure Unix and Linux operating systems on various hardware platforms.
2. To administer and manage Unix/Linux systems effectively, including system monitoring and performance tuning.
3. To understand the basics of Unix/Linux, including command-line operations and system architecture.
4. To manage users, create shell scripts, and perform text processing tasks using Unix/Linux commands.
5. To master package management tools in Unix/Linux for software installation, updates, and removals.
6. To use basic networking commands for network configuration and troubleshooting in Unix/Linux.
7. To develop advanced shell scripting skills for tasks such as regular expressions, job scheduling, and log file analysis.

Course Content

Program No.	Program Statement	T	P	S
1	To understand the Unix/Linux file system hierarchy, navigate the system, and practice managing files and directories and how to manage users and groups in a Linux environment and perform user modification tasks.		2	
2	To understand and manage file permissions using chmod and chown commands and essential Linux commands for file management, system monitoring, and automation.		2	
3	To learn how to use the vim command-line text editor for creating, editing, saving, and closing text files and practice using the cat command for opening and concatenating multiple text files.		2	
4	To understand and apply piping and redirection techniques to manipulate the flow of data between commands and use text processing tools like grep, sed, and awk to filter, manipulate, and extract data from text files.		2	
5	To explore the grep command for searching and filtering text based on specific patterns and practice using the sed command to substitute and manipulate text within files.		2	
6	To learn how to use the awk command to extract and process specific data from text files.		2	
7	To understand and implement the basic syntax, variables, and operators in the Bash shell, and write a Bash script to check if a file exists or not with and without control statement.		2	
8	To practice input/output redirection by writing a Bash script that redirects input and output between commands and files and combine commands using pipelines and filters in Bash script that chains commands efficiently to manipulate data.		2	
9	To write and execute simple shell scripts that automate tasks using various Bash programming techniques, including file manipulation and system operations.		2	
10	To learn how to install and remove software packages using package managers like apt (for Debian and Ubuntu) and yum/dnf (for Fedora) commands and practice creating symlinks (both hard and soft) using the ln command.		2	
11	To learn and apply file system management techniques by creating, mounting, and unmounting file systems using the mount and umount commands.		2	
12	To use regular expressions in shell commands like find, grep, sed, and awk to match patterns and process text efficiently by searching for error messages in system logs, extracting IP addresses from log files, and modifying log file content.		2	
13	To schedule jobs using cron to automate recurring tasks at specified intervals and perform log file analysis using tools like logrotate and grep to manage and analyze system logs for maintenance and troubleshooting.		2	
14	Configure a system's network interfaces and assign an IP address using commands such as ip, netstat, netctl, ipconfig, or ifconfig. Verify network connectivity by pinging google.com and 8.8.8.8. Configure DNS resolution by editing the host.conf file and use hostnamectl and host commands to resolve the domain name google.com to its IP address.		2	
15	Connect to a remote system securely using SSH (e.g., ssh user@remote_host.com). Transfer a file named sample.txt from the local system to a remote system using SCP,		2	

	and vice versa using FTP. Set up a basic firewall rule using ufw to allow traffic on port 22 from the IP range 192.168.1.0/24. Create a new user, modify the user details, assign file ownership and permissions using chmod and chown, and then delete the user after verification.			

Scheme of End Semester Examination: As per DSEU Regulation - 2(A), 2024 – End Term Examination: 70 Marks, Continuous Evaluation: 30 Marks, Total: 100 Marks.

Recommended Books and References:

1. Unix Concepts and Applications, Sumitabha Das, McGraw Hill, 4th Edition
2. Advanced Programming in the UNIX Environment, W. Richard Stevens, Pearson Education, 3rd Edition
3. Unix and Shell Programming, Behrouz A. Forouzan, Richard F. Gilberg, Cengage Learning (Thomson)
4. Linux System Programming, Robert Love, O'Reilly Media, SPD
5. The Linux Programming Interface: A Linux and UNIX System Programming Handbook, Michael Kerrisk, No Starch Press
6. Classic Shell Scripting: Hidden Commands that Unlock the Power of Unix, Arnold Robbins and Nelson H.F. Beebe, O'Reilly Media
7. The Linux Command Line: A Complete Introduction, William E. Shotts Jr., No Starch Press
8. Learning the Bash Shell, Cameron Newham, O'Reilly Media

Learning outcomes:

1. Students will be able to do essential Linux/Unix operations such as navigating the file system, managing users and groups, and modifying file permissions using command-line tools.
2. Students will be able to do shell scripting using Bash to automate system-level tasks with the use of variables, control structures, loops, and redirection.
3. Students will be able to do text processing using tools like grep, sed, and awk for filtering, transforming, and extracting data from text files.
4. Students will be able to do system administration tasks including package management, disk usage monitoring, file system mounting/unmounting, and task scheduling with cron.
5. Students will be able to do basic network configuration and troubleshooting using commands like ip, ping, and netstat, along with secure remote connections using SSH, and file transfers using FTP and SCP.
6. Students will be able to do advanced Linux/Unix tasks such as using regular expressions in scripts, managing processes with tools like ps and top, and analyzing logs using grep and logrotate.

Hyperlinks of suggested e-Resources:

1. Linux workshop 10 days (https://linuxessentials.netlify.app)
2. Guided Self learning (https://linuxjourney.com)
3. The man pages online (https://man7.org/linux/man-pages/)
4. The Linux foundation (https://training.linuxfoundation.org/)
5. https://www.coursera.org/learn/hands-on-introduction-to-linux-commands-and-shell-scripting

Pedagogical approach:

- Classroom teaching using board work or audio-visual aids, depending on the requirement of the topic.
- Flipped classroom methodology to encourage active engagement and pre-class preparation.
- Quizzes and presentations can be conducted periodically to reinforce understanding and assess application, as deemed necessary by the instructor.

Preferred Tools

- Oracle VirtualBox: To install and operate Linux-based environments for hands-on practice and lab activities.

Additional information (if any)

Course Title: Web Development			
Type of Course: DCC	**Level of Course: 5.0**	**Delivery Sub Type of the course:**	**Theory**
Course code: CSE-4-403-T	**No. of credits: 2**	**T-P-S: 2-0-0**	**Learning hours: 30**

Pre-requisite and Co-requisite of Course: DBMS, SQL and MongoDB, Programming Knowledge

Department: Computer Science Engineering

Course Objective :

1. To understand and apply the foundational elements of web development using HTML, CSS, and JavaScript.
2. To build well-structured and accessible web pages using semantic HTML tags.
3. To design responsive layouts using modern CSS techniques including Flexbox, Grid, and media queries.
4. To develop interactive front-end applications using modern JavaScript (ES.Next) and DOM manipulation.
5. To build dynamic user interfaces using React, incorporating components, state management, and hooks.
6. To design and implement RESTful APIs using Node.js and Express.js.
7. To perform CRUD operations and handle data storage using MongoDB.
8. To integrate frontend and backend components to create full-stack web applications.

Course Content

Module/ Unit	**Topic**	**T**	**P**	**S**
1	**Introduction to Web Development and HTML Basics :** Web Pages, Static and dynamic web pages, client-server architecture, client-side and server-side scripting languages, Basic structure of an HTML webpage (DOCTYPE, html, head, body), meta tags, comments, text formatting, lists, tables, forms, semantic tags, image tag, anchor tag, basic attributes, character entities.	5		
2	**CSS Fundamentals and Responsive Design:** CSS syntax, selectors, properties, box model (margin, padding, border, content), positioning (static, relative, absolute, fixed, sticky), Flexbox, CSS Grid, media queries, responsive web design, background images, colors, fonts, DOM manipulation.	5		
3	**JavaScript Essentials and ES.Next :** JavaScript syntax, variables (var, let, const), data types (String, Number, Object, Array), operators, loops, conditional statements, functions, scope, objects, events, DOM manipulation, arrow functions, de-structuring, template literals, import/export modules, JSON, fetch API.	6		
4	**React Fundamentals and Backend with Node.js :** Introduction to React and Angular UI, Setting up React environment using Create React App, components (functional and class), JSX syntax, props, state management, event handling, React Hooks (useState, useEffect), forms, conditional rendering, Node.js setup, Express.js server, HTTP methods (GET, POST, PUT, DELETE), connecting React with Node.js (API calls).	9		
5	**MongoDB, REST APIs, and Web Security :** MongoDB, Mongoose, CRUD operations, creating RESTful APIs, Express.js routes, web security concepts (CORS, JWT, XSS, CSRF), protecting API routes, authentication middleware.	5		
6	**Testing, Deployment, and Project :** Testing React components with Jest, testing Node.js routes with Mocha, deployment on Netlify/Vercel (React), deployment on Heroku (Node.js), environment variables, full-stack application project (React + Node.js + MongoDB), deploying full-stack web applications.	4		

Scheme of End Semester Examination: As per DSEU Regulation - 2(A), 2024 – End Term Examination: 70 Marks, Continuous Evaluation: 30 Marks, Total: 100 Marks.

Recommended Books and References:

1. HTML and CSS: Design and Build Websites by Jon Duckett, 1st Edition, Wiley, 2011.
2. CSS: The Definitive Guide by Eric A. Meyer and Estelle Weyl, 5th Edition, O'Reilly Media, 2023.
3. HTML & CSS: The Complete Reference by Thomas A. Powell, 5th Edition, McGraw Hill, 2010.
4. JavaScript and JQuery: Interactive Front-End Web Development by Jon Duckett, 1st Edition, Wiley, 2014.
5. Eloquent JavaScript: A Modern Introduction to Programming by Marijn Haverbeke, 3rd Edition, No Starch Press, 2018.
6. Learning React: Functional Web Development with React and Redux by Alex Banks and Eve Porcello, 2nd Edition, O'Reilly Media, 2020.
7. Node.js Design Patterns: Design and Implement Production-Grade Applications by Mario Casciaro and Luciano Mammino, 3rd Edition, Packt Publishing, 2020.
8. MongoDB: The Definitive Guide: Powerful and Scalable Data Storage by Shannon Bradshaw, Eoin Brazil, and Kristina Chodorow, 3rd Edition, O'Reilly Media, 2019.
9. Full-Stack React, TypeScript, and Node by David Choi, 1st Edition, Packt Publishing, 2022.
10. React Quickly by Morten Barklund, Azat Mardan, 2nd Edition, Manning, 2023..

Learning outcomes:
students will be able to

1. Develop structured, accessible web pages and design responsive layouts using HTML, CSS, and JavaScript, while leveraging ES.Next features to enhance user interactions and functionality.
2. Build dynamic and reusable components using React, effectively managing state and handling user events.
3. Design and implement RESTful APIs and manage server-side logic with Node.js and Express.js to support frontend functionality.
4. Integrate frontend and backend technologies seamlessly to build cohesive, full-stack web applications, ensuring smooth communication between client-side and server-side components.
5. Implement client-side form validation, error handling, and interactive UI elements to improve the user experience and ensure data integrity.
6. Manage data storage and retrieval with MongoDB, performing CRUD operations and integrating the database with backend applications.
7. Utilize modern version control practices with Git to manage and collaborate on web development projects.
8. Understand and apply security best practices in web development, including input sanitization and secure API design.
9. Optimize web applications for performance, ensuring fast loading times and efficient use of resources.
10. Deploy full-stack web applications to cloud platforms or servers, ensuring they are accessible and functional in a production environment..

Hyperlinks of suggested e-Resources:

1. Introduction to Modern Application Development | https://archive.nptel.ac.in/courses/106/106/106106156/
2. Modern Application Development | https://archive.nptel.ac.in/courses/106/106/106106222/
3. Introduction to Web Development | https://www.coursera.org/learn/web-development
4. Full Stack Web Development https://www.coursera.org/learn/fullstack-web-development
5. Web Design for Everybody: Basics of Web Development & Coding Specialization | https://www.coursera.org/specializations/web-design
6. Software Engineering for Web Applications (6.171) | https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-171-software-engineering-for-web-applications-fall-2003/
7. Web Frameworks | Software Studio (6.170) | https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-170-software-studio-spring-2013/resources/mit6_170s13_12-web-frmwks/

Pedagogical approach

- Hands-on Learning: Emphasize practical coding sessions to reinforce theoretical concepts.
- Project-Based Approach: Build real-world projects to integrate and apply skills from various modules.
- Scaffolded Content: Progress from foundational to advanced topics in a structured manner.
- Interactive Demonstrations: Use live coding and tools to demonstrate concepts in real-time.
- Collaborative Exercises: Encourage teamwork through group activities and version control practice.
- Industry-Relevant Tools: Introduce modern frameworks, libraries, and deployment platforms.

Additional information (if any)

Course Title: Web Development Practice			
Type of Course: DCC	**Level of Course: 5.0**	**Delivery Sub Type of the course: Practical**	
Course Code: CSE-4–404-P	**No. of credits: 1**	**T-P-S: 0-2-0**	**Learning hours: 30**

Pre-requisite and Co-requisite of Course: DBMS, SQL and MongoDB, Programming Knowledge

Department: Computer Science Engineering

Course objectives

1. To Use Web Development Fundamentals to Create static web pages using HTML, CSS, and JavaScript.
2. To Build Accessible Web Pages: Apply semantic HTML tags to improve SEO and readability.
3. To Implement Responsive Design: Use CSS techniques like flexbox, grid, and media queries for adaptable layouts across devices.
4. To Develop Interactive Applications: Leverage ES.Next features and manipulate the DOM with JavaScript for dynamic interactions.
5. To Master React: Build dynamic UIs with React, focusing on components, state, hooks, and lifecycle methods.
6. To Design RESTful APIs: Implement efficient APIs with Node.js and Express.js for client-server communication.
7. To Practice CRUD with MongoDB: Perform Create, Read, Update, and Delete operations on data in MongoDB.
8. To Integrate Full-Stack Development: Combine React, Node.js, Express, and MongoDB to create a fully functional web application.

Course Content

Program No.	**Program Statement**	**T**	**P**	**S**
1	Create a static webpage to display all text formatting tags in HTML, showcasing text styling like bold, italic, underline, and headings.		2	
2	Design a personal profile webpage for a classroom or school, using semantic HTML tags to structure personal information, such as name, bio, contact details, and an image.		2	
3	Create a webpage for an online contact form, where users can fill out their details, including name, email, and message, using HTML form elements.		2	
4	Style the previously created contact form webpage using an external CSS stylesheet, focusing on layout, form control design, and responsive behavior.		2	
5	Design a personal profile webpage for a student or faculty member with customized styles for typography, color scheme, and layout using flexbox and grid systems for responsiveness.		2	
6	Write a JavaScript program to demonstrate the use of conditional statements and loops, simulating an interactive quiz or decision-based process.		2	
7	Create a webpage that displays the use of arrays and strings in JavaScript, where the content updates dynamically based on user input or selections.		2	
8	Develop a form validation page where users can input their email, password, and phone number, and JavaScript validates them before submission.		2	
9	Create a counter webpage with increment, decrement, and reset functionality, where users can track a number (e.g., daily goals or countdowns) and interact with it.		2	
10	Build a React app that displays a product catalog or a list of courses offered by a university, using reusable components to manage each product or course.		2	
11	Design a simple React counter app where users can increase or decrease the count and reset it to zero, utilizing React's useState hook.		2	
12	Create a React app that fetches data from an external source (e.g., a list of university departments or faculty members) and updates the page dynamically using the useEffect hook.		2	
13	Create a webpage that provides instructions on downloading and installing Node.js, with step-by-step guidance for setting up a development environment.		2	
14	Develop a web server in Node.js that serves static HTML, CSS, and JavaScript files, allowing users to access content hosted on a local server.		2	
15	Write a Node.js script that reads from and writes to a file, simulating a simple logging system for user activity or a school attendance record system.		2	

Scheme of End Semester Examination: As per DSEU Regulation - 2(A), 2024 – End Term Examination: 70 Marks, Continuous Evaluation: 30 Marks, Total: 100 Marks.

Recommended Books and References:

1. HTML and CSS: Design and Build Websites by Jon Duckett, 1st Edition, Wiley, 2011.
2. CSS: The Definitive Guide by Eric A. Meyer and Estelle Weyl, 5th Edition, O'Reilly Media, 2023.
3. HTML & CSS: The Complete Reference by Thomas A. Powell, 5th Edition, McGraw Hill, 2010.
4. JavaScript and JQuery: Interactive Front-End Web Development by Jon Duckett, 1st Edition, Wiley, 2014.
5. Eloquent JavaScript: A Modern Introduction to Programming by Marijn Haverbeke, 3rd Edition, No Starch Press, 2018.

6. Learning React: Functional Web Development with React and Redux by Alex Banks and Eve Porcello, 2nd Edition, O'Reilly Media, 2020.
7. Node.js Design Patterns: Design and Implement Production-Grade Applications by Mario Casciaro and Luciano Mammino, 3rd Edition, Packt Publishing, 2020.
8. MongoDB: The Definitive Guide: Powerful and Scalable Data Storage by Shannon Bradshaw, Eoin Brazil, and Kristina Chodorow, 3rd Edition, O'Reilly Media, 2019.
9. Full-Stack React, TypeScript, and Node by David Choi, 1st Edition, Packt Publishing, 2022.
10. React Quickly by Morten Barklund, Azat Mardan, 2nd Edition, Manning, 2023..

Learning outcomes:

1. Students will be able to create static web pages that showcase HTML text formatting tags, and they will be able to design structured webpages using semantic HTML tags for improved accessibility and SEO.
2. Students will be able to design personal profile pages and implement HTML forms to collect user inputs such as contact details, integrating external style sheets for improved visual appeal and layout.
3. Students will be able to apply advanced CSS techniques such as flexbox and grid systems to style webpages, and will learn to create responsive, mobile-friendly layouts.
4. Students will be able to write JavaScript to handle conditional statements, loops, and array manipulation, as well as create form validation scripts to ensure correct input.
5. Students will be able to build React applications with reusable components, manage state using hooks like useState, and implement side effects with useEffect for fetching external data.
6. Students will be able to set up Node.js servers, build basic RESTful APIs for CRUD operations, and integrate MongoDB for storing and managing user data.

Hyperlinks of suggested e-Resources:

1. **MDN Web Docs - Web Technologies** | URL: https://developer.mozilla.org/en-US/docs/Web
2. **W3Schools - Web Development Tutorials** | URL: https://www.w3schools.com/
3. **JavaScript.info - JavaScript Tutorial** | URL: https://javascript.info/
4. **Eloquent JavaScript - A Modern Introduction to Programming** | URL: https://eloquentjavascript.net/
5. **React Documentation** | URL: https://react.dev/
6. **Node.js Documentation** | URL: https://nodejs.org/en
7. **Git Documentation** | https://git-scm.com/doc

Pedagogical approach

- Hands-on Learning: Emphasize practical coding sessions to reinforce theoretical concepts.
- Project-Based Approach: Build real-world projects to integrate and apply skills from various modules.
- Scaffolded Content: Progress from foundational to advanced topics in a structured manner.
- Interactive Demonstrations: Use live coding and tools to demonstrate concepts in real-time.
- Collaborative Exercises: Encourage teamwork through group activities and version control practice.
- Industry-Relevant Tools: Introduce modern frameworks, libraries, and deployment platforms.

Additional information (if any)

Course Title: Python Programming Practice			
Type of Course: DCC	**Level of Course: 5.0**	**Delivery Sub Type of the course: Practical**	
Course Code: CSE-4-405-P	**No. of credits: 1**	**T-P-S: 0-2-0**	**Learning hours: 30**

Prerequisite and Corequisite of Course:

1. **Logical Thinking**: This is essential because programming is about solving problems logically. Having an understanding of how to break down problems into smaller, manageable steps is crucial for writing code.
2. **Basic Mathematics**: While deep mathematical knowledge isn't necessary, understanding basic concepts such as variables, simple operations (addition, subtraction, multiplication, division), and logical operators is helpful.
3. **Familiarity with Basic Computer Usage**: An understanding of how to use a computer and interact with an IDE (Integrated Development Environment) or text editor is necessary to start programming.

Department: Computer Science Engineering

Course objectives

1. To install and set up Python interpreters (IDLE, PyCharm, Conda) in both Linux and Windows environments.
2. To learn the syntax, constructs, data types, and data structures of the Python language.
3. To understand and implement control structures, including loops and conditionals, in Python.
4. To gain proficiency in using functions and modular programming to write efficient Python code.
5. To master object-oriented programming principles, including classes, objects, inheritance, and polymorphism.
6. To learn and apply Python packages such as Pandas, NumPy, Matplotlib, and Seaborn for advanced data analysis and visualization.

Course Content

Module /Unit	**Topic**	**T**	**P**	**S**
1	Introduction to Python programming language and its ecosystem, installation of Python on Linux/Windows using native package managers (e.g., APT for Ubuntu), installing Python using Conda and PyCharm on Linux/Windows, setting up Python development environments using iPython terminal and Jupyter notebooks, using Google Colab for cloud-based Python development, understanding Python basic data types like integer, float, string, list, tuple, set, and dictionary, variable assignment and manipulation, using collections (list, tuple, set, dictionary), defining and calling functions in Python, using loops (for, while), conditional statements (if, elif, else), iterating over data structures, using core libraries like NumPy for numerical computations, Pandas for data manipulation, Matplotlib and Seaborn for data visualization, and writing Python programs to perform calculations (e.g., area of geometric shapes) and plotting results. Program Statements: 1. Write a Python program to install Python on Linux/Windows using the native package manager (e.g., APT on Ubuntu) and verify the installation. 2. Write a Python program to calculate the area of a circle, rectangle, and square using basic mathematical formulas. 3. Write a Python program that demonstrates the use of basic data types (integer, float, string, list, tuple, set, and dictionary) by performing operations on them. 4. Write a Python program that uses loops (for, while) and conditional statements (if, elif, else) to iterate through a list and display even numbers. 5. Write a Python program that uses NumPy and Matplotlib to generate random data and plot a histogram of the data.		4	
2	Introduction to Python syntax, understanding basic constructs and operators, using the print() function to display output, learning different print formatting specifiers and styles to format output, taking user input using the input() function and storing it in variables, demonstrating the use of arithmetic, comparison, logical, and relational operators, using conditional statements to check conditions like positive or negative numbers, and writing Python programs to check divisibility by 3, 5, or both. Program Statements: 6. Write a Python program that prints a greeting message using the print() function. 7. Write a Python program that takes the user's name and age as input and displays a personalized greeting message. 8. Write a Python program that demonstrates the use of arithmetic operators (addition, subtraction, multiplication, division). 9. Write a Python program to check if a number is positive or negative using conditional statements.		4	

	10. Write a Python program that checks if a number is divisible by 3, 5, or both using conditional statements.			
3	Understanding loops and functions in Python, using for and while loops for iteration, performing tasks such as calculating squares, summing numbers, and excluding specific values from a list, defining and calling functions to perform operations like summing two numbers and using lambda functions for one-liner operations, working with immutable (tuple) and mutable (list, dictionary, set) objects, demonstrating the use of global and local variables, using recursion to calculate factorials, utilizing dictionary and list comprehensions for efficient data processing, and performing set operations like union, difference, and intersection. Additionally, checking for the existence of elements in lists, sets, and dictionaries, and working with tuples, dictionaries, and sets to perform data manipulations. Program Statements: 11. Write a Python program that uses a for loop to print the square of each number in a given list of integers. 12. Write a Python program that uses a while loop to sum all the numbers from 1 to a given number n (inclusive). 13. Write a Python program that prints all numbers from 1 to 9 except 5 using a loop. 14. Write a Python function that takes two numbers as arguments and returns their sum. 15. Write a Python program that checks if a specific element exists in a list, set, and dictionary.		8	
4	Object-Oriented Programming (OOP) concepts in Python, including Encapsulation, Inheritance, and Abstraction, are essential for understanding how to structure and model real-world data. These concepts allow you to define classes, objects, and methods while controlling the accessibility and behavior of attributes and functions. Exception handling in Python is crucial for managing errors and unexpected situations in code using try, except, and finally blocks. Custom exceptions can be created for more specialized error handling. File handling in Python involves reading, writing, editing, and closing files in both text and binary modes, as well as using the pickle module to serialize and deserialize objects for storage. Program Statements: 16. Create a Python class with private variables, and implement getter and setter methods to access and modify these private variables. This ensures controlled access to the class attributes. 17. Create a base class with basic attributes and methods, then create a derived class that inherits the properties and behaviors of the base class. The derived class can override or extend the functionality of the inherited methods. 18. Define an abstract base class with one or more abstract methods. Implement a subclass that provides specific implementations for these abstract methods. This hides the complexity of the implementation from the user. 19. Implement a Python program with try, except, and finally blocks. Handle common exceptions such as division by zero, invalid input, and file not found. The finally block ensures that cleanup operations occur, regardless of whether an exception was raised. 20. Create a custom exception by defining a class that inherits from Python's Exception class. Use this custom exception in your program to raise errors under specific conditions (e.g., invalid input or business logic errors). 21. Open a file in text mode, write data into the file, read its contents, and modify the data. After the operations, close the file to ensure that all changes are saved and the file handle is properly released. 22. Open a file in binary mode, write binary data into it (e.g., image or non-text files), read its contents, and close the file. This mode is used for handling non-text files that require specific encoding. 23. Use the pickle module to serialize and save Python objects (such as lists or dictionaries) to a file. Later, load the objects back into the program using pickle.load(). This allows for saving complex data structures in a file. 24. Use the pickle module to open a saved pickle file, deserialize the Python objects, and retrieve them in their original form. This operation is useful when dealing with complex data that needs to be stored for future use.		8	
5	Write programs to read, edit, save, and manage data in CSV and Excel files using Pandas. Perform sorting, filtering, and subset selection in Pandas DataFrames, including adding, deleting, and handling missing values (NaN). Create new columns based on calculations from existing data using the apply() method. Use NumPy to create arrays with random values, zeros, and ones, and perform operations like transposition,		6	

	multiplication, and dot/cross products. Utilize Pandas, Matplotlib, and Seaborn for data visualization, including generating bar charts, line charts, pie charts, histograms, and scatter plots. Program statement : 25. Create a Pandas DataFrame, generate a new column by applying a function to existing columns, and then plot the data using Matplotlib to visualize the results in a line chart and a bar chart. 26. Create NumPy arrays filled with random values, zeros, and ones, and perform matrix operations like transpose, multiplication, dot product, and cross product on these arrays. 27. Use Seaborn and Matplotlib to generate a variety of visualizations: a pie chart, histogram, and scatter plot, based on data from a Pandas DataFrame, while customizing labels, titles, and colors. 28. Use Pandas to filter and manipulate data, including sorting, selecting subsets of rows and columns, and adding or deleting columns, and then visualize relationships between multiple variables using a pair plot in Seaborn. 29. Save a Matplotlib figure as an image file (PNG or JPEG) after customizing the plot's appearance, including adjusting labels, colors, and styles, and use Pandas to handle the data visualization process efficiently.			

Scheme of End Semester Examination: As per DSEU Regulation - 2(A), 2024 – End Term Examination: 70 Marks, Continuous Evaluation: 30 Marks, Total: 100 Marks.

Recommended Books and References:

1. Programming Python, Mark Lutz, O'Reilly Media, Inc., 1st Edition, 2001.
2. Concepts of Programming Languages, Robert W. Sebesta, Pearson Education India, 7th Edition, 2004.
3. Introduction to Programming Using Python, Y. Daniel Liang, Pearson, 3rd Edition, 2013.
4. Exploring Python, Timothy Budd, McGraw Hill Higher Education, 1st Edition, 2010.
5. Python Data Analytics with Pandas, NumPy, and Matplotlib, Fabio Nelli, 2018.

Learning outcomes:

Students will be able to:

1. Understand and apply basic data structures such as lists, sets, dictionaries, and tuples in Python for efficient problem-solving.
2. Develop reusable code for various real-life scenarios, enhancing code modularity and maintainability.
3. Perform file handling operations, including reading, writing, and modifying data in text, binary, and pickle formats.
4. Analyze large datasets using Pandas and NumPy for data manipulation, cleaning, and transformation.
5. Visualize data with Matplotlib and Seaborn, creating different types of plots to derive meaningful insights.
6. Implement and optimize algorithms to manipulate data efficiently, improving the performance of Python applications.

Hyperlinks of suggested e-Resources:

1. **Coursera- Python for Everybody Specialization https://www.coursera.org/specializations/python**
2. **Coursera- Data Analysis with Python https://www.coursera.org/learn/data-analysis-with-python**
3. **NPTEL - The Joy of Computing using Python https://onlinecourses.nptel.ac.in/noc24_cs57/preview**

Pedagogical approach :

- Lecture-based teaching for theoretical concepts.
- Problem-solving sessions to apply algorithms.
- Use of visualization tools like VisuAlgo.
- Hands-on coding assignments.
- Flipped classroom for discussions and exercises.
- Group activities for collaborative learning.
- Gamification through competitive coding challenges.
- Case studies on real-world algorithm applications.

Additional Information (If any)

Course Title: Discrete Mathematical Structures			
Type of Course: DSE	**Level of Course: 5.0**	**Delivery Sub Type of the course:**	**Theory**
Course code: CSE-4-407-T	**No. of credits: 4**	**T-P-S: 4-0-0**	**Learning hours: 60**

Pre-requisite and Co-requisite of Course: Basic mathematics of 12th standard

Department: Computer Science Engineering

Course objectives

1. To get familiar and understand the fundamental notions in discrete mathematics
2. To understand and demonstrate the basic concept of an algorithm and its application in combinatorial mathematics
3. To identify the basic properties of graphs and trees and model simple applications

Course content

Module /Unit	Topic	T	P	S
1	Set theory and Mathematical Logic: Sets, posets, venn diagrams, set operations, cardinality of sets, multi sets, countable and uncountable sets. Statements and notations, Connectives, Well formed formulas, Truth Tables, tautology, equivalence implication, Normal forms, Quantifiers, universal quantifiers. Predicative logic, Free & Bound variables, Rules of inference, Consistency, proof of contradiction, Automatic Theorem Proving. Probability – sample space, conditional probability, expectation, linearity of expectation, variance, Markov, Chebychev, probabilistic methods.	15		
2	Relations and Algebraic Structures: Functions (mappings / transformations), Properties of Binary Relations, equivalence, closures, compatibility and partial ordering relations, Lattices, Hasse diagram. Inverse Function Composition of functions, recursive Functions, Lattice and its Properties, Algebraic systems Examples and general properties, Semigroups and monads, cyclic groups, sub groups', Abelian groups, homomorphism, Isomorphism. Concepts of Rings and Fields. Galois theory.	15		
3	Graph Theory: Definitions And Basic Results - Representation of A Graph By A Matrix And Adjacency List - Trees - Cycles - Properties - Paths and Connectedness – Subgraphs, DFS, BFS, Spanning Trees, planar Graphs. Graph Theory and Applications, Basic Concepts Isomorphism and Sub graphs, Multi graphs and Euler circuits, Hamiltonian graphs, Chromatic Numbers.	15		
4	Number Theory: Divisibility and Modular Arithmetic, primes and greatest common divisors, congruences and their solutions, applications of congruences, Euclidean algorithm, Finite fields of form GF(p), Polynomial Arithmetic, Finite fields of the form GF(2n), prime numbers, Fermat's and Euler's theorems, testing of primality, primitive roots, discrete logarithms.	15		

Scheme of End Semester Examination: As per DSEU Regulation - 2(A), 2024 – End Term Examination: 70 Marks, Continuous Evaluation: 30 Marks, Total: 100 Marks.

Recommended Books and References:

1. *Discrete Mathematical Structures with Applications to Computer Science*, J.P. Tremblay and R. Manohar, Tata McGraw Hill, 2001
2. *Discrete Mathematics & its Applications*, K. H. Rosen, Tata McGraw-Hill, 6th Edition, 2007
3. *Elements of Discrete Mathematics*, C. L. Liu, Tata McGraw-Hill, 2nd Edition, 2000
4. *Introductory Discrete Mathematics*, V. K. Balakrishnan, Dover Publications, 1996
5. *Discrete Structures, Logic, and Computability*, J. L. Hein, Jones and Bartlett, 3rd Edition, 2010

Learning Outcomes: Upon completion of the course, students will be able:

1. Ability to distinguish between the notion of discrete and continuous mathematical structures
2. Ability to construct and interpret finite state diagrams and DFSA
3. Application of induction and other proof techniques towards problem solving

Hyperlinks of suggested e-Resources:

1. https://onlinecourses.nptel.ac.in/noc22_cs33/preview
2. https://und.edu/academics/online/enroll-anytime/math208.html
3. https://www.youtube.com/playlist?list=PLHXZ9OQGMqxersk8fUxiUMSIx0DBqsKZS
4. https://ocw.mit.edu/courses/18-310-principles-of-discrete-applied-mathematics-fall-2013/
5. https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-fall-2010/

6. https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/
Pedagogical approach: • Classroom teaching on board or audio visual medium as and when necessary • Flip classroom Quiz, presentation (As felt necessary by the teacher)
Additional information (if any)

Course Title: Computational Mathematics			
Type of Course: DSE	**Level of Course: 5.0**	**Delivery Sub Type of the course:**	**Theory**
Course code: CSE-4-406-T	**No. of credits: 4**	**T-P-S:4-0-0**	**Learning hours: 60**

Pre-requisite and Co-requisite of Course: Basics of Set Theory and Calculus

Department: Computer Science Engineering

Course objectives: To give understanding about the basics of probability and various concepts of statistics

Course Content

Module/ Unit	Topic	T	P	S
1	Probability and Distribution: Classical, relative frequency and axiomatic definitions of probability, addition rule and conditional probability, multiplication rule, total probability, Bayes' Theorem and independence, problems.	12		
2	Random variables (Discrete and Continuous Random variable) Probability mass function and Probability density function, Expectation and variance, Discrete and Continuous Probability distribution: Binomial, Poission and Normal distributions.	12		
3	Introduction: Measures of central tendency, Moments, Moment generating function (MGF) , Skewness, Kurtosis, Curve Fitting , Method of least squares, Fitting of straight lines, Fitting of second degree parabola, Exponential curves ,Correlation and Rank correlation	12		
4	Regression Analysis: Regression lines of y on x and x on y, regression coefficients, properties of regressions coefficients and non linear regression. Sampling, Testing of Hypothesis and Statistical Quality Control: Introduction , Sampling Theory (Small and Large) , Hypothesis, Null hypothesis, Alternative hypothesis, Testing a Hypothesis,	12		
5	Level of significance, Confidence limits, Test of significance of difference of means, T-test, F-test and Chi-square test, One way Analysis of Variance (ANOVA).	12		

Scheme of End Semester Examination: As per DSEU Regulation - 2(A), 2024 – End Term Examination: 70 Marks, Continuous Evaluation: 30 Marks, Total: 100 Marks.

Recommended Books and References:

1. An Introduction to Probability and Statistics, V.K. Rohatgi & A.K. Md. E. Saleh, Wiley Publishers, Latest Edition.
2. Introduction to Probability and Statistics, J.S. Milton & J.C. Arnold, McGraw Hill Education, Latest Edition.
3. A First Course in Probability, Sheldon Ross, Pearson Education, 10th Edition.

Learning outcomes

Students will be able to:

1. Calculate and interpret measures of central tendency, correlation, regression, and their properties.
2. Apply basic probability concepts and understand discrete and continuous probability distributions.
3. Conduct hypothesis testing and analyze the results using appropriate statistical methods.
4. Use control charts to monitor quality control and assess process stability
5. Analyze data samples effectively to draw meaningful conclusions.
6. Communicate statistical findings through clear and concise visualizations and reports.

Hyperlinks of suggested e-Resources:

1. **https://www.slideshare.net/slideshow/statistics-probability-75922344/75922344**
2. **https://www.geeksforgeeks.org/probability-and-statistics/**

Pedagogical approach:

- Classroom teaching on board or audio visual medium as and when necessary
- Flip classroom Quiz, presentation (As felt necessary by the teacher)

Additional information (if any)

Course Title: Programming in Java			
Type of Course: OEC	**Level of Course: 5.0**	**Delivery Sub Type of the course:**	**Theory**
Course code: CSE-4-306-T	**No. of credits: 3**	**T-P-S : 3-0-0**	**Learning hours: 45**

Pre-requisite and Co-requisite of Course: Basics of Computer Programming

Department: Computer Science Engineering

Course objectives

1. Understand Core Java Concepts
2. Application of Object-Oriented Programming Practices

Course content

Module/ Unit	Topic	T	P	S
1	Introduction: History and evolution of Java, Overview and characteristics of JAVA and OOP, JVM, JRE and JDK, Java Program structure, variables, arrays and data types, Java Tokens, access modifiers, Operators and Control Structure, Garbage Collection, Security Architecture and Security Policy.	5		
2	Classes and Instances, class member modifiers, wrapper class, inner class, Interfaces and Abstract Classes. Abstract methods, encapsulation, inheritance, polymorphism, Dynamic Binding, information hiding.	5		
3	Constructor and its types. Polymorphism - Varieties of Polymorphism, Polymorphic Variables, Constructor Overloading, Method Overloading, Method Overriding, Abstract Methods, Pure Polymorphism, Virtual	5		
4	String Class, StringBuffer Class, String Manipulation methods, string operations (Concatenation, comparison, substring extraction, etc.)	5		
5	Input/Output Operations Reading and writing to files, File streams (FileInputStream, FileOutputStream, FileReader, FileWriter), Cosole I/O using Scanner class and System.out.println().	5		
6	Exception Handelling: Exception Handling in Java – Introduction to exception handling, Types of exceptions, try-catch blocks, Multiple catch blocks, Nested try blocks, Custom exceptions, Finally block, Throw and Throws keywords, Handling ArrayIndexOutOfBoundsException and NullPointerException.	5		
7	Multi-Threading Fundamentals, Thread class, Runnable Interface, the Life Cycle of Thread, Creation of single and multiple threads.	5		
8	Introduction to applets. Life cycle of applet and security concerns. The AWT – The AWT Class Hierarchy, The Layout Manager, User Interface Components, Panels, Dialogs, The Menu Bar, Introduction to Servlets.	5		
9	Swing GUI Design – Introduction to Swing, Components (JTextField, JComboBox, JButton), Event Handling using ActionListener, Designing Student Registration Form, Connecting GUI with MySQL using JDBC, Inserting and Retrieving Data.	5		

Scheme of End Semester Examination: As per DSEU Regulation - 2(A), 2024 – End Term Examination: 70 Marks, Continuous Evaluation: 30 Marks, Total: 100 Marks.

Recommended Books and References:

1. Herbert Schildt, “Java 2 Complete Reference”, TMH.
2. E. Bala Guruswamy, “Programming with Java”, TMH.

Learning outcomes (note: outcomes will be mapped with evaluation criteria and modules)

1. Enhancement of Programming Skills.
2. Ability to design and implement robust and secure applications using Java.

Hyperlinks of suggested e-Resources:

1. **https://www.w3schools.com/**
2. **https://www.geeksforgeeks.org/**
3. **https://www.tutorialspoint.com/**

Pedagogical approach:

- Classroom teaching on board or audio visual medium as and when necessary
- Flip classroom Quiz, presentation (As felt necessary by the teacher)

Additional information (if any)

Course Title: Programming in Java Practice			
Type of Course: OEC	**Level of Course: 5.0**	**Delivery Sub Type of the course:**	**Practical**
Course code: CSE-4-307-P	**No. of credits: 1**	**T-P-S: 0-2-0**	**Learning hours: 30**

Pre-requisite and Co-requisite of Course: Basics of Computer Programming

Department: Computer Science Engineering

Course objectives

1. Design and implement Java Applications.
2. Application of advanced Java Concepts.

Course Content

Program No.	Program Statement	T	P	S
1	Install and configure the Java Development Kit (JDK), set up environment variables, and install and configure an IDE such as IntelliJ IDEA or Eclipse.		2	
2	Write a program to implement basic array operations and stack functionality using arrays and methods.		2	
3	Write a program to demonstrate the use of conditional statements: if, if-else, and switch.		2	
4	Write a program using loop constructs: for, while, and do-while to perform repetitive tasks.		2	
5	Create a class-based system to manage student records, including name, roll number, and marks using methods and objects.		2	
6	Develop a banking application with operations like account creation, deposit, withdrawal, and balance check using parameterized and default constructors.		2	
7	Build a class hierarchy of shapes (Circle, Rectangle, Triangle) and apply single, multilevel, and hierarchical inheritance, and demonstrate runtime polymorphism.		2	
8	Extend the banking system to implement method overloading and overriding to handle different transaction types.		2	
9	Enhance the banking system using final, static, super, and abstract keywords for class members and inheritance control.		2	
10	Write a program to create and use a user-defined package containing utility classes.		2	
11	Create a program to demonstrate operations on String and StringBuffer: concatenation, comparison, and substring extraction.		2	
12	Create a package that implements dynamic polymorphism and interfaces; demonstrate usage of wrapper classes like Integer.		2	
13	Implement exception handling with try, catch, finally, throw, and throws; handle common exceptions such as ArrayIndexOutOfBoundsException and NullPointerException; include a user-defined exception.		2	
14	Write a program that demonstrates thread creation using both Thread class and Runnable interface and manages thread lifecycle methods.		2	
15	Develop a GUI-based calculator application using Swing components with event handling and basic arithmetic operations.		2	

Scheme of End Semester Examination: As per DSEU Regulation - 2(A), 2024 – End Term Examination: 70 Marks, Continuous Evaluation: 30 Marks, Total: 100 Marks.

Recommended Books and References:

1. **Herbert Schildt, "Java 2 Complete Reference", TMH.**
2. **E. Bala Guruswamy, "Programming with Java", TMH.**

Learning outcomes (note: outcomes will be mapped with evaluation criteria and modules)

1. Enhancement of Programming Skills.
2. Understanding Object Oriented Programming.
3. Ability to design and implement robust and secure applications using Java.

Hyperlinks of suggested e-Resources:

1. **https://www.w3schools.com/**
2. **https://www.geeksforgeeks.org/**
3. **https://www.tutorialspoint.com/**

Pedagogical approach:

Additional information (if any)